

Google Sheet Template Tabs

The Google Sheet file contains these tabs:

- `sources` (Input)
- `category_whitelist`
- `topic_dictionary`
- `employee_groups`
- `jurisdiction_overrides`
- `normalized_laws` (Output)
- `suggested_topics`

Goal

আগে Google Sheet-এ stable normalized output তৈরি হবে। API/DB integration পরে করবো।

1) Input Sheet: `sources`

Each source row contains:

- `source_id` (unique key)
- `source_name`
- `source_type` (rss/api/html)
- `feed_url_or_base_url`
- `priority`, `active`, `rate_limit_ms`
- Defaults (optional):
 `primary_category_default`, `topic_default`, `jurisdiction_level_default`,
 `state_code_default`, `employee_group_default`

✓ Only process rows where `active=1`.

2) Controlled taxonomy (must be stable)

- **Primary category is strict:** AI must choose only from `category_whitelist` (10 keys).
- **Topic is controlled:** AI must choose `topic_key` only from `topic_dictionary` under the selected primary category.
- If no topic matches: set `topic_key=null` and add a row to `suggested_topics` (dedupe).

This prevents AI drift and keeps filters consistent for future tools (comparison/dashboard/handbook/templates).

3) Employee groups (controlled)

- Use `employee_groups` tab as the controlled list.
 - If `employee_group_default` is given in `sources`, map to a valid `employee_group_key`. Otherwise keep blank.
-

4) Jurisdiction overrides (implement in workflow)

- Read active rules from `jurisdiction_overrides` (supports wildcard `*`).
- After AI categorization, apply override logic for the same (`state_code + primary_category + topic_key`):

MVP rule: if a state rule exists for the same group, federal becomes superseded.

✓ In output, include:

- `override_rule_applied`
- `is_superseded` (true/false)
- `superseded_by_post_url` (optional)

(If you prefer not to remove rows, just mark superseded rows; later UI/tools can filter `is_superseded=false`.)

5) Workflow Steps (n8n)

For each active source:

1. Scrape RSS/API/HTML
2. Normalize: `title`, `post_url`, `published_at`, `raw_content`
3. Clean content + generate `content_hash`
4. Lookup `normalized_laws` by `post_url` (or hash) to decide new/updated/skipped
5. AI classify:
 - `primary_category` (whitelist)
 - `topic_key` (topic_dictionary)

- optional `tags`, `summary`, `confidence`
 - 6. Apply `jurisdiction_overrides` (mark superseded where applicable)
 - 7. Final JSON node (Code node) → output one structured JSON per item
 - 8. Upsert into `normalized_laws`
 - 9. If AI suggests a new topic → insert into `suggested_topics` (dedupe)
-

6) Output Sheet: `normalized_laws`

Write these fields per item:

- Identity: `source_id`, `source_name`, `source_type`, `base_url`
 - Unique: `post_url`
 - Content: `title`, `published_at`, `effective_date`, `content_text`, `content_html`
 - Filters: `jurisdiction_level`, `state_code`, `employee_group`
 - Taxonomy: `primary_category`, `topic_key`, `tags`
 - Quality: `summary`, `confidence`
 - Dedup: `content_hash`
 - Tracking: `status`, `error_message`, `first_seen_at`, `last_seen_at`
 - Overrides: `override_rule_applied`, `is_superseded`, `superseded_by_post_url`
-

7) Final JSON Node location

Place the Final JSON Code node right before writing to Google Sheet:

Scrape → Parse/Extract → Normalize → Hash → AI Categorize → Overrides →  Final JSON → Upsert Output

Output must be **one item = one JSON object** matching the `normalized_laws` columns.

Example “Final JSON Node” output (shape) It should produce exactly these keys (matches your `normalized_laws` sheet columns):

```
{
  "source_id": "dol_rss_001",
  "source_name": "U.S. Department of Labor",
  "source_type": "rss",
  "base_url": "https://www.dol.gov",
  "post_url": "https://example.gov/article/123",
  "title": "Minimum wage update for 2026",
  "published_at": "2026-02-16T08:00:00Z",
  "effective_date": "2026-03-01",
```

```
{
  "jurisdiction_level": "federal",
  "state_code": null,
  "employee_group": null,
  "primary_category": "wages_compensation",
  "topic_key": "minimum_wage",
  "tags": "minimum_wage;pay_transparency",
  "summary": "Short summary...",
  "content_text": "Cleaned text...",
  "content_html": "<p>Clean HTML...</p>",
  "confidence": 0.86,
  "content_hash": "abcd1234",
  "status": "new",
  "error_message": "",
  "first_seen_at": "2026-02-23T10:00:00Z",
  "last_seen_at": "2026-02-23T10:00:00Z"
}
```

Implementation of **jurisdiction_overrides** in n8n

Goal

When both **federal** and **state** laws exist for the same **primary_category + topic_key**, the workflow should apply override rules and output only the correct “winner” row(s) into **normalized_laws**.

Workflow placement (where to apply overrides)

Apply overrides **after**:

- normalization
- hashing/dedup
- AI classification (primary_category + topic_key)
...and **before** final “Upsert normalized_laws”.

So:

Scrape → **Normalize** → **Hash** → **AI Categorize** → **(Group + Apply Overrides)** → **Final JSON** → **Upsert**

Minimum data needed for overrides (must exist in the Final JSON)

Each item must have:

- `jurisdiction_level` (federal/state/local)
- `state_code` (nullable)
- `primary_category`
- `topic_key`
- `effective_date` (optional but helps)
- `post_url` (unique)

These already match your spec/sheet.

Override rules supported (simple set)

Use `jurisdiction_overrides.rule_type` with these values:

1. **state_overrides_federal**
If a state record exists for the same (`primary_category`, `topic_key`, `state_code`), prefer the state record over the federal record.
2. **use_state_only**
Same as above but stricter: if state exists, **drop federal completely**.
3. **stricter_wins** (*optional now*)
Hard to compute without a “strictness score”. For MVP, treat it same as `state_overrides_federal` (or skip implementing this until you add a strictness scoring field).

Your sheet already includes these rule ideas.

How to read the overrides from Google Sheet

Add a Google Sheets “Read” node:

- Tab: `jurisdiction_overrides`
- Filter: `active = 1`

Treat rows with `*` as wildcards:

- `state_code="*"` applies to all states
- `primary_category_key="*"` applies to all categories
- `topic_key="*"` applies to all topics

How the matching should work

For each law item, find applicable override rule(s) by:

- Match `state_code` exactly OR *
- Match `primary_category` exactly OR *
- Match `topic_key` exactly OR *

Choose the **most specific rule** if multiple match:

Specificity order:

1. exact state + exact category + exact topic
2. exact state + exact category + *
3. exact state + * + *
4. * + * + * (default)

The actual override logic (easy and reliable)

Do overrides **per group**:

Group key:

- If state law: `state_code|primary_category|topic_key`
- If federal: also evaluate it against each state it affects (MVP approach below)